

Multi-Task Learning with Vision Transformers: Joint Classification and Saliency Prediction

Nicholas Welsh

Florida Institute of Technology
Department of Computer Science & Mathematics

March 2026

Outline

- 1 Motivation
- 2 Model
- 3 Data
- 4 Experiment
- 5 Results
- 6 Conclusion

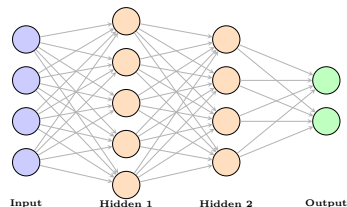
From Classical ML to Deep Learning

- So far we have seen models like KNN, decision trees, and logistic regression.
- These work well on **structured, tabular data** (spreadsheets with rows and columns).
- But images are not tables. A 224×224 color image has over 150,000 raw pixel values. Classical models struggle with this.
- **Neural networks** solve this by automatically learning which patterns in those pixels matter, instead of us hand-picking features.

This project: we use a neural network called a Vision Transformer to process images directly.

What Is a Neural Network?

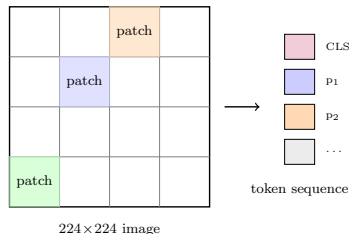
- Made of layers of simple units (“neurons”) connected together.
- Each layer transforms its input and passes it to the next.
- Early layers detect simple patterns (edges, colors). Later layers combine them into complex ones (eyes, objects).
- A **deep** network has many layers, so it can learn very complex patterns.



Multi-Layer Perceptron (MLP)

What Is a Vision Transformer?

- A **Vision Transformer (ViT)** is a neural network for images.
- Instead of a sliding window, it:
 - ① Cuts the image into a grid of **patches**.
 - ② Processes all patches at once.
- **Attention** lets the model decide which patches matter.
- We use **DeiT-Small** (Data-efficient Image Transformer), pretrained on 1.2M images.



Why Multi-Task Learning?

- Most models do one thing: “Is this a dog?”
- **Multi-task learning** trains one model to do multiple things at once.
- Related tasks share useful patterns, so learning one can help the other.
- Benefits: more efficient (one model, not two) and can improve accuracy through shared features.



```
{"labels": ["dog", "fire_hydrant"], "saliency": "14x14  
heatmap"}
```

Our Two Tasks

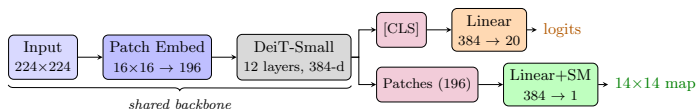
Task 1: Classification

- “What objects are in this image?”
- Labels: 20 common categories (person, dog, car, etc.)
- An image can contain multiple objects (multi-label).

Task 2: Saliency Prediction

- “Where do humans look in this image?”
- Ground truth: actual gaze data from thousands of people.
- Output: a heatmap showing predicted gaze density.

How the Model Works



- The image is split into 196 patches (14×14 grid).
- All patches are processed together by the transformer backbone.
- Two separate “heads” read the output:
 - **Classification head:** looks at a summary token (CLS) $\rightarrow$ predicts object categories.
 - **Saliency head:** looks at each patch $\rightarrow$ predicts how likely people are to look there.

What Is Fine-Tuning?

- DeiT-Small was already trained on 1.2 million images to recognize 1,000 object types.
- It already “knows” how to see edges, textures, shapes, and objects.
- Instead of training from scratch, we **fine-tune**: we take this pretrained model and continue training it on our specific data.
- This is much faster and works better than starting from zero, especially when we have limited data (10,000 images).

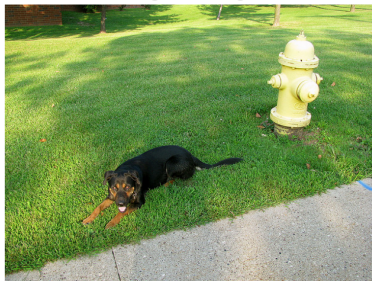
Analogy: teaching someone who already speaks English to read medical papers is easier than teaching someone who has never seen text.

Our Datasets

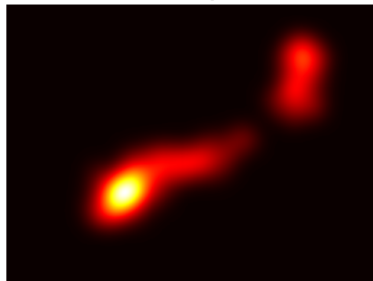
- **SALICON**: 10,000 training images + 5,000 validation images from COCO.
 - Each image comes with a **fixation map**: a heatmap showing where thousands of people looked.
 - Collected by tracking mouse movements (approximates eye tracking).
- **COCO 2014**: the same images, but with labels for what objects are present (person, dog, car, etc.).
- By pairing these two datasets, each image has both:
 - Category labels (“dog, person”) for classification.
 - A gaze heatmap for saliency prediction.

What the Data Looks Like

Original Image



Human Gaze Map (SALICON)



Left: the original photo. Right: the **SALICON** fixation map, a heatmap showing where thousands of people looked when viewing this image. Bright spots mean more people looked there. This is our ground truth for the saliency task.

How We Train the Model

- The model sees each training image and tries to:
 - ① Predict which objects are present (classification).
 - ② Predict where people look (saliency).
- After each prediction, we compute a **loss**: a number that measures how wrong the model was.
- The model adjusts its internal parameters to reduce the loss.
- This process repeats for 15 **epochs** (passes through the full training set).
- We save the version of the model that performed best on a held-out validation set.

What Is a Loss Function?

- A loss function is how we tell the model “how wrong you are.”
- For classification: **binary cross-entropy**. Penalizes the model for being confident about wrong labels.
- For saliency, we tested two options:
 - **MSE**: average squared difference between prediction and ground truth.
 - **KL (Kullback–Leibler) divergence**: measures how different two probability distributions are.
- This choice turned out to be the most important design decision in the project.

$$\mathcal{L} = \underbrace{\text{BCE}(\hat{y}, y)}_{\text{classification}} + \lambda \cdot \underbrace{\mathcal{L}_{\text{sal}}}_{\text{saliency}} \quad \mathcal{L}_{\text{sal}} = \begin{cases} \text{MSE}(p, \hat{p}) \\ \text{KL}(p \parallel \hat{p}) \end{cases}$$

What We Compare Against

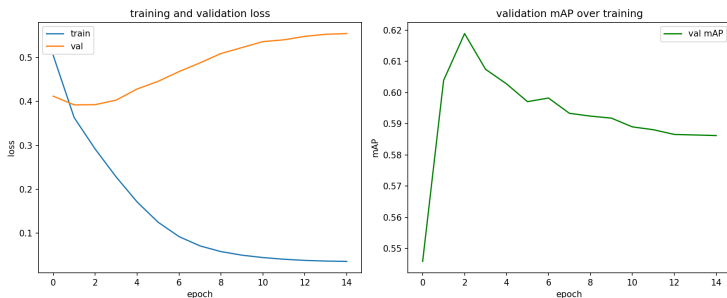
To know if our model is any good, we need baselines:

- **Frozen-feature baselines:** use the pretrained model as a fixed feature extractor (no fine-tuning) and train simple classifiers on top.
 - Logistic regression (a simple linear classifier)
 - MLP (a small 1-layer neural network)
- **Single-task ablations:** train with only one task active to see how multi-tasking compares.
 - **cls-only:** only classification, no saliency.
 - **sal-only:** only saliency, no classification.

How We Measure Success

- **Classification: mAP** (mean average precision)
 - Measures how well the model ranks correct labels above incorrect ones.
 - 1.0 = perfect, 0.0 = random guessing.
- **Saliency: CC** (correlation coefficient)
 - Measures how well the predicted heatmap matches the human gaze heatmap.
 - 1.0 = perfect match, 0.0 = no relationship.
- We also report SIM (distribution overlap) and KL divergence (distributional distance).

Training Curves



- Left: loss decreases over training (model gets better).
- Right: classification accuracy on validation data.
- The model peaks early (epoch 2–3), then starts to overfit (memorize training data). We save the best early version.

Classification: Fine-Tuning Helps

Method	mAP
Logistic regression (frozen)	0.556
MLP classifier (frozen)	0.566
ViT cls-only (fine-tuned)	0.623
ViT multitask (fine-tuned)	0.604

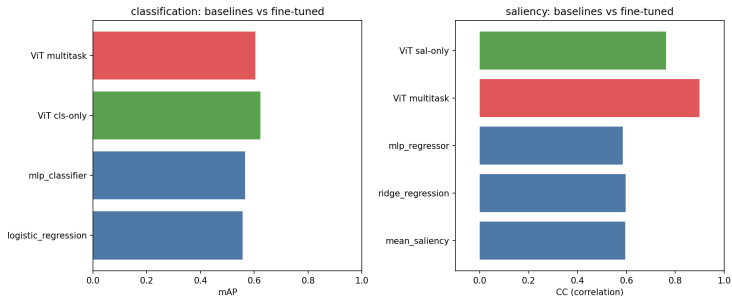
- Fine-tuning improves over frozen baselines by 5–6 points.
- Multi-task is slightly worse than classification-only (0.604 vs 0.623).
- That 1.9-point gap is the “cost” of sharing the model with saliency.

The Big Surprise: Loss Function Matters More Than Architecture

Saliency Loss	mAP	CC	SIM	KL
MSE	0.612	0.196	0.430	1.087
KL divergence	0.604	0.899	0.799	0.182

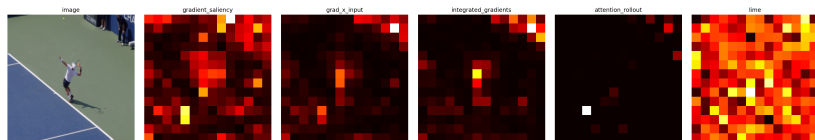
- Same model, same data, same everything. Only the loss function changed.
- Saliency correlation jumps from 0.196 to 0.899.
- **Why?** MSE treats the output as raw numbers. KL divergence treats it as a probability distribution, which is what our model actually outputs.
- Lesson: how you measure “wrongness” matters as much as the model itself.

Visual Comparison



- Left: classification accuracy. Fine-tuned models (red/green) beat frozen baselines (blue).
- Right: saliency correlation. Fine-tuned models (green) substantially beat baselines (blue).

What the Model “Sees”



A tennis player. The model's saliency head predicts where people look (bright = high attention). The prediction concentrates on the player, matching human gaze patterns.

What We Learned

- ① **One model, two tasks.** A single Vision Transformer can classify images and predict human gaze at the same time, with only a small accuracy trade-off.
- ② **Fine-tuning beats simple baselines.** Adapting a pretrained model to our data outperforms simpler approaches that use the same model as a fixed feature extractor.
- ③ **Loss function design is critical.** Switching from MSE to KL divergence raised saliency correlation from 0.196 to 0.899 without changing anything else. Choosing the right way to measure error can matter more than choosing the right model.

Why This Matters

- In practice, many real-world problems involve multiple related tasks. Multi-task learning lets you build more efficient systems.
- The loss function finding is a cautionary tale: a model can appear to fail when the real problem is how you are training or evaluating it.
- Understanding these interactions between model design, loss functions, and evaluation metrics is a core skill in applied machine learning.

Limitations

- Results are from a single training run (seed 42). Running multiple times with different random seeds would give more confidence.
- KL and MSE losses have different scales, which also changes how the two tasks are balanced. A more thorough study would sweep across different balance settings.
- Our saliency maps are low resolution (14×14 patches), so they are not directly comparable to high-resolution saliency benchmarks.

Questions?

Nicholas Welsh
nwelsh2024@my.fit.edu

github.com/nickvpn/vit-ml-xai-project